Chapter 2. Creating and Running Containers

Kubernetes is a platform for creating, deploying, and managing distributed applications. These applications come in many different shapes and sizes, but ultimately, they are all comprised of one or more programs that run on individual machines. These programs accept input, manipulate data, and then return the results. Before we can even consider building a distributed system, we must first consider how to build the *application container images* that contain these programs and make up the pieces of our distributed system.

Application programs are typically comprised of a language runtime, libraries, and your source code. In many cases, your application relies on external shared libraries such as `libc` and `libssl`. These external libraries are generally shipped as shared components in the OS that you have installed on a particular machine.

This dependency on shared libraries causes problems when an application developed on a programmer's laptop has a dependency on a shared library that isn't available when the program is rolled out to the production OS. Even when the development and production environments share the exact same version of the OS, problems can occur when developers forget to include dependent asset files inside a package that they deploy to production.

The traditional methods of running multiple programs on a single machine require that all of these programs share the same versions of shared libraries on the system. If the different programs are developed by different teams or organizations, these shared dependencies add needless complexity and coupling between these teams.

A program can only execute successfully if it can be reliably deployed onto the machine where it should run. Too often the state of the art for deployment involves running imperative scripts, which inevitably have twisty and byzantine failure cases. This makes the task of rolling out a new version of all or parts of a distributed system a labor-intensive and difficult task.

In Chapter 1, we argued strongly for the value of immutable images and infrastructure. This immutability is exactly what the container image provides. As we will see, it easily

solves all the problems of dependency management and encapsulation just described.

When working with applications, it's often helpful to package them in a way that makes sharing them with others easy. Docker, the default tool most people use for containers, makes it easy to package an executable and push it to a remote registry where it can later be pulled by others. At the time of writing, container registries are available in all of the major public clouds, and services to build images in the cloud are also available in many of them. You can also run your own registry using open source or commercial systems. These registries make it easy for users to manage and deploy private images, while image-builder services provide easy integration with continuous delivery systems.

For this chapter, and the remainder of the book, we are going to work with a simple example application that we built to help show this workflow in action. You can find the application on GitHub.

Container images bundle a program and its dependencies into a single artifact under a root filesystem. The most popular container image format is the Docker image format, which has been standardized by the Open Container Initiative to the OCI image format. Kubernetes supports both Docker- and OCI-compatible images via Docker and other runtimes. Docker images also include additional metadata used by a container runtime to start a running application instance based on the contents of the container image.

This chapter covers the following topics:

- How to package an application using the Docker image format
- How to start an application using the Docker container runtime

Container Images

For nearly everyone, their first interaction with any container technology is with a container image. A *container image* is a binary package that encapsulates all of the files necessary to run a program inside of an OS container. Depending on how you first experiment with containers, you will either build a container image from your local filesystem or download a preexisting image from a *container registry*. In either case, once the container image is present on your computer, you can run that image to produce a running application inside an OS container.

The most popular and widespread container image format is the Docker image format, which was developed by the Docker open source project for packaging, distributing, and running containers using the `docker` command. Subsequently, work has begun by

Docker, Inc., and others to standardize the container image format via the Open Container Initiative (OCI) project. While the OCI standard achieved a 1.0 release milestone in mid-2017, adoption of these standards is proceeding slowly. The Docker image format continues to be the de facto standard and is made up of a series of filesystem layers. Each layer adds, removes, or modifies files from the preceding layer in the filesystem. This is an example of an *overlay* filesystem. The overlay system is used both when packaging up the image and when the image is actually being used. During runtime, there are a variety of different concrete implementations of such filesystems, including `aufs`, `overlay`, and `overlay2`.

CONTAINER LAYERING

The phrases “Docker image format” and “container images” may be a bit confusing. The image isn’t a single file but rather a specification for a manifest file that points to other files. The manifest and associated files are often treated by users as a unit. The level of indirection allows for more efficient storage and transmittal. Associated with this format is an API for uploading and downloading images to an image registry.

Container images are constructed with a series of filesystem layers, where each layer inherits and modifies the layers that came before it. To help explain this in detail, let's build some containers. Note that for correctness, the ordering of the layers should be bottom up, but for ease of understanding, we take the opposite approach:

```
.
└── container A: a base operating system only, such as Debian
    └── container B: build upon #A, by adding Ruby v2.1.10
    └── container C: build upon #A, by adding Golang v1.6
```

At this point we have three containers: A, B, and C. B and C are *forked* from A and share nothing besides the base container’s files. Taking it further, we can build on top of B by adding Ruby on Rails (version 4.2.6). We may also want to support a legacy application that requires an older version of Ruby on Rails (e.g., version 3.2.x). We can build a container image to support that application based on B also, planning to someday migrate the app to version 4:

```
. (continuing from above)
└── container B: build upon #A, by adding Ruby v2.1.10
    └── container D: build upon #B, by adding Rails v4.2.6
    └── container E: build upon #B, by adding Rails v3.2.x
```

Conceptually, each container image layer builds upon a previous one. Each parent reference is a pointer. While the example here is a simple set of containers, other real-world containers can be part of a larger extensive directed acyclic graph.

Container images are typically combined with a container configuration file, which provides instructions on how to set up the container environment and execute an application entry point. The container configuration often includes information on how to set up networking, namespace isolation, resource constraints (cgroups), and what `syscall` re-

strictions should be placed on a running container instance. The container root filesystem and configuration file are typically bundled using the Docker image format.

Containers fall into two main categories:

- System containers
- Application containers

System containers seek to mimic virtual machines and often run a full boot process. They often include a set of system services typically found in a VM, such as `ssh`, `cron`, and `syslog`. When Docker was new, these types of containers were much more common. Over time, they have come to be seen as poor practice and application containers have gained favor.

Application containers differ from system containers in that they commonly run a single program. While running a single program per container might seem like an unnecessary constraint, it provides the perfect level of granularity for composing scalable applications and is a design philosophy that is leveraged heavily by Pods. We will examine how Pods work in detail in Chapter 5.

Building Application Images with Docker

In general, container orchestration systems like Kubernetes are focused on building and deploying distributed systems made up of application containers. Consequently, we will focus on application containers for the remainder of this chapter.

Dockerfiles

A Dockerfile can be used to automate the creation of a Docker container image.

Let's start by building an application image for a simple Node.js program. This example would be very similar for many other dynamic languages, like Python or Ruby.

The simplest of npm/Node/Express apps has two files: *package.json* (Example 2-1) and *server.js* (Example 2-2). Put these in a directory and then run `npm install express --save` to establish a dependency on Express and install it.

Example 2-1. package.json

```
{
  "name": "simple-node",
  "version": "1.0.0",
  "description": "A sample simple application for Kubernetes Up & Running",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "author": ""
}
```

Example 2-2. server.js

```
var express = require('express');

var app = express();
app.get('/', function (req, res) {
  res.send('Hello World!');
});
app.listen(3000, function () {
  console.log('Listening on port 3000!');
  console.log('  http://localhost:3000');
});
```

To package this up as a Docker image, create two additional files: *.dockerignore* (Example 2-3) and the Dockerfile (Example 2-4). The Dockerfile is a recipe for how to build the container image, while *.dockerignore* defines the set of files that should be ignored when copying files into the image. A full description of the syntax of the Dockerfile is available on the Docker website.

Example 2-3. .dockerignore

```
node_modules
```

Example 2-4. Dockerfile

```
# Start from a Node.js 16 (LTS) image ❶
FROM node:16
```

```
# Specify the directory inside the image in which all commands will run ❷
WORKDIR /usr/src/app

# Copy package files and install dependencies ❸
COPY package*.json ./
RUN npm install
RUN npm install express

# Copy all of the app files into the image ❹
COPY . .

# The default command to run when starting the container ❺
CMD [ "npm", "start" ]
```

❶ Every Dockerfile builds on other container images. This line specifies that we are starting from the `node:16` image on the Docker Hub. This is a preconfigured image with Node.js 16.

❷ This line sets the work directory in the container image for all following commands.

❸ These three lines initialize the dependencies for Node.js. First, we copy the package files into the image. This will include *package.json* and *package-lock.json*. The `RUN` command then runs the correct command *in the container* to install the necessary dependencies.

❹ Now we copy the rest of the program files into the image. This will include everything except *node_modules*, as that is excluded via the *.dockerignore* file.

❺ Finally, we specify the command that should be run when the container is run.

Run the following command to create the `simple-node` Docker image:

```
$ docker build -t simple-node .
```

When you want to run this image, you can do it with the following command. Navigate to *http://localhost:3000* to access the program running in the container:

```
$ docker run --rm -p 3000:3000 simple-node
```

At this point, our `simple-node` image lives in the local Docker registry where the image was built and is only accessible to a single machine. The true power of Docker comes from the ability to share images across thousands of machines and the broader Docker community.

Optimizing Image Sizes

There are several gotchas people encounter when they begin to experiment with container images that lead to overly large images. The first thing to remember is that files that are removed by subsequent layers in the system are actually still present in the images; they're just inaccessible. Consider the following situation:

```
.
└── layer A: contains a large file named 'BigFile'
    └── layer B: removes 'BigFile'
        └── layer C: builds on B by adding a static binary
```

You might think that *BigFile* is no longer present in this image. After all, when you run the image, it is no longer accessible. But in fact it is still present in layer A, which means that whenever you push or pull the image, *BigFile* is still transmitted through the network, even if you can no longer access it.

Another pitfall revolves around image caching and building. Remember that each layer is an independent delta from the layer below it. Every time you change a layer, it changes every layer that comes after it. Changing the preceding layers means that they need to be rebuilt, repushed, and repulled to deploy your image to development.

To understand this more fully, consider two images:

```
.
└── layer A: contains a base OS
    └── layer B: adds source code server.js
        └── layer C: installs the 'node' package
```

versus:

```
.
└── layer A: contains a base OS
```

```
└── layer B: installs the 'node' package
    └── layer C: adds source code server.js
```

It seems obvious that both of these images will behave identically, and indeed the first time they are pulled, they do. However, consider what happens when *server.js* changes. In the second case, it is only that change that needs to be pulled or pushed, but in the first case, both *server.js* and the layer providing the `node` package need to be pulled and pushed, since the `node` layer is dependent on the *server.js* layer. In general, you want to order your layers from least likely to change to most likely to change in order to optimize the image size for pushing and pulling. This is why, in Example 2-4, we copy the *package*.json* files and install dependencies before copying the rest of the program files. A developer is going to update and change the program files much more often than the dependencies.

Image Security

When it comes to security, there are no shortcuts. When building images that will ultimately run in a production Kubernetes cluster, be sure to follow best practices for packaging and distributing applications. For example, don't build containers with passwords baked in—and this includes not just in the final layer, but any layers in the image. One of the counterintuitive problems introduced by container layers is that deleting a file in one layer doesn't delete that file from preceding layers. It still takes up space, and it can be accessed by anyone with the right tools—an enterprising attacker can simply create an image that only consists of the layers that contain the password.

Secrets and images should *never* be mixed. If you do so, you will be hacked, and you will bring shame to your entire company or department. We all want to be on TV someday, but there are better ways to go about that.

Additionally, because container images are narrowly focused on running individual applications, a best practice is to minimize the files within the container image. Every additional library in an image provides a potential vector for vulnerabilities to appear in your application. Depending on the language, you can achieve very small images with a very tight set of dependencies. This smaller set ensures that your image isn't exposed to vulnerabilities in libraries it would never use.

Multistage Image Builds

One of the most common ways to accidentally build large images is to do the actual program compilation as part of the construction of the application container image. Compiling code as part of the image build feels natural, and it is the easiest way to build a container image from your program. The trouble with doing this is that it leaves all of the unnecessary development tools, which are usually quite large, lying around inside your image and slowing down your deployments.

To resolve this problem, Docker introduced *multistage builds*. With multistage builds, rather than producing a single image, a Docker file can actually produce multiple images. Each image is considered a stage. Artifacts can be copied from preceding stages to the current stage.

To illustrate this concretely, we will look at how to build our example application, `kuard`. This is a somewhat complicated application that involves a React.js frontend (with its own build process) that then gets embedded into a Go program. The Go program runs a backend API server that the *React.js* frontend interacts with.

A simple Dockerfile might look like this:

```
FROM golang:1.17-alpine

# Install Node and NPM
RUN apk update && apk upgrade && apk add --no-cache git nodejs bash npm

# Get dependencies for Go part of build
RUN go get -u github.com/jteeuwen/go-bindata/...
RUN go get github.com/tools/godep
RUN go get github.com/kubernetes-up-and-running/kuard

WORKDIR /go/src/github.com/kubernetes-up-and-running/kuard

# Copy all sources in
COPY . .

# This is a set of variables that the build script expects
ENV VERBOSE=0
ENV PKG=github.com/kubernetes-up-and-running/kuard
ENV ARCH=amd64
ENV VERSION=test
```

```
# Do the build. This script is part of incoming sources.
RUN build/build.sh

CMD [ "/go/bin/kuard" ]
```

This Dockerfile produces a container image containing a static executable, but it also contains all of the Go development tools and the tools to build the *React.js* frontend and the source code for the application, neither of which are needed by the final application. The image, across all layers, adds up to over 500 MB.

To see how we would do this with multistage builds, examine the following multistage Dockerfile:

```
# STAGE 1: Build
FROM golang:1.17-alpine AS build

# Install Node and NPM
RUN apk update && apk upgrade && apk add --no-cache git nodejs bash npm

# Get dependencies for Go part of build
RUN go get -u github.com/jteeuwen/go-bindata/...
RUN go get github.com/tools/godep

WORKDIR /go/src/github.com/kubernetes-up-and-running/kuard

# Copy all sources in
COPY . .

# This is a set of variables that the build script expects
ENV VERBOSE=0
ENV PKG=github.com/kubernetes-up-and-running/kuard
ENV ARCH=amd64
ENV VERSION=test

# Do the build. Script is part of incoming sources.
RUN build/build.sh

# STAGE 2: Deployment
FROM alpine

USER nobody:nobody
COPY --from=build /go/bin/kuard /kuard
```

```
CMD [ "/kuard" ]
```

This Dockerfile produces two images. The first is the *build* image, which contains the Go compiler, *React.js* toolchain, and source code for the program. The second is the *deployment* image, which simply contains the compiled binary. Building a container image using multistage builds can reduce your final container image size by hundreds of megabytes and thus dramatically speed up your deployment times, since generally, deployment latency is gated on network performance. The final image produced from this Dockerfile is somewhere around 20 MB.

These scripts are present in the `kuard` repository on GitHub and you can build and run this image with the following commands:

```
# Note: if you are running on Windows you may need to fix line-endings using:
# --config core.autocrlf=input
$ git clone https://github.com/kubernetes-up-and-running/kuard
$ cd kuard
$ docker build -t kuard .
$ docker run --rm -p 8080:8080 kuard
```

Storing Images in a Remote Registry

What good is a container image if it's only available on a single machine?

Kubernetes relies on the fact that images described in a Pod manifest are available across every machine in the cluster. One option for getting this image to all machines in the cluster would be to export the `kuard` image and import it on each of them. We can't think of anything more tedious than managing Docker images this way. The process of manually importing and exporting Docker images has human error written all over it. Just say no!

The standard within the Docker community is to store Docker images in a remote registry. There are tons of options when it comes to Docker registries, and what you choose will be largely based on your needs in terms of security and collaboration features.

Generally speaking, the first choice you need to make regarding a registry is whether to use a private or a public registry. Public registries allow anyone to download images

stored in the registry, while private registries require authentication to download images. In choosing public versus private, it's helpful to consider your use case.

Public registries are great for sharing images with the world because they allow for easy, unauthenticated use of the container images. You can easily distribute your software as a container image and have confidence that users everywhere will have the exact same experience.

In contrast, a private registry is best for storing applications that are private to your service and that you don't want the world to use. Additionally, private registries often provide better availability and security guarantees because they are specific to you and your images rather than serving the world.

Regardless, to push an image, you need to authenticate to the registry. You can generally do this with the `docker login` command, though there are some differences for certain registries. In the examples in this book we are pushing to the Google Cloud Platform registry, called the Google Container Registry (GCR); other clouds, including Azure and Amazon Web Services (AWS), also have hosted container registries. For new users hosting publicly readable images, the Docker Hub is a great place to start.

Once you are logged in, you can tag the `kuard` image by prepending the target Docker registry. You can also append an identifier that is usually used for the version or variant of that image, separated by a colon (`:`):

```
$ docker tag kuard gcr.io/kuar-demo/kuard-amd64:blue
```

Then you can push the `kuard` image:

```
$ docker push gcr.io/kuar-demo/kuard-amd64:blue
```

Now that the `kuard` image is available on a remote registry, it's time to deploy it using Docker. When we pushed the image to GCR, it was marked as public, so it will be available everywhere without authentication.

The Container Runtime Interface

Kubernetes provides an API for describing an application deployment, but relies on a container runtime to set up an application container using the container-specific APIs na-

tive to the target OS. On a Linux system that means configuring cgroups and namespaces. The interface to this container runtime is defined by the Container Runtime Interface (CRI) standard. The CRI API is implemented by a number of different programs, including the `containerd-cri` built by Docker and the `cri-o` implementation contributed by Red Hat. When you install the Docker tooling, the `containerd` runtime is also installed and used by the Docker daemon.

Starting with release 1.25 of Kubernetes, only container runtimes that support the CRI will work with Kubernetes. Fortunately, managed Kubernetes providers have made this transition nearly automatic for users of managed Kubernetes.

Running Containers with Docker

In Kubernetes, containers are usually launched by a daemon on each node called the *kubelet*; however, it's easier to get started with containers using the Docker command-line tool. The Docker CLI tool can be used to deploy containers. To deploy a container from the `gcr.io/kuar-demo/kuard-amd64:blue` image, run the following command:

```
$ docker run -d --name kuard \
  --publish 8080:8080 \
  gcr.io/kuar-demo/kuard-amd64:blue
```

This command starts the `kuard` container and maps ports 8080 on your local machine to 8080 in the container. The `--publish` option can be shortened to `-p`. This forwarding is necessary because each container gets its own IP address, so listening on *localhost* inside the container doesn't cause you to listen on your machine. Without the port forwarding, connections will be inaccessible to your machine. The `-d` option specifies that this should run in the background (daemon), while `--name kuard` gives the container a friendly name.

Exploring the kuard Application

`kuard` exposes a simple web interface, which you can load by pointing your browser at *http://localhost:3000* or via the command line:

```
$ curl http://localhost:8080
```

`kuard` also exposes a number of interesting functions that we will explore later on in this book.

Limiting Resource Usage

Docker enables applications to use fewer resources by exposing the underlying cgroup technology provided by the Linux kernel. These capabilities are likewise used by Kubernetes to limit the resources each Pod uses.

Limiting memory resources

One of the key benefits to running applications within a container is the ability to restrict resource utilization. This allows multiple applications to coexist on the same hardware and ensures fair usage.

To limit `kuard` to 200 MB of memory and 1 GB of swap space, use the `--memory` and `--memory-swap` flags with the `docker run` command.

Stop and remove the current `kuard` container:

```
$ docker stop kuard
$ docker rm kuard
```

Then start another `kuard` container using the appropriate flags to limit memory usage:

```
$ docker run -d --name kuard \
  --publish 8080:8080 \
  --memory 200m \
  --memory-swap 1G \
  gcr.io/kuar-demo/kuard-amd64:blue
```

If the program in the container uses too much memory, it will be terminated.

Limiting CPU resources

Another critical resource on a machine is the CPU. Restrict CPU utilization using the `--cpu-shares` flag with the `docker run` command:

```
$ docker run -d --name kuard \
  --publish 8080:8080 \
  --memory 200m \
  --memory-swap 1G \
  --cpu-shares 1024 \
  gcr.io/kuar-demo/kuard-amd64:blue
```

Cleanup

Once you are done building an image, you can delete it with the `docker rmi` command:

```
docker rmi <tag-name>
```

or:

```
docker rmi <image-id>
```

Images can either be deleted via their tag name (e.g., `gcr.io/kuar-demo/kuard-amd64:blue`) or via their image ID. As with all ID values in the `docker` tool, the image ID can be shortened as long as it remains unique. Generally only three or four characters of the ID are necessary.

It's important to note that unless you explicitly delete an image, it will live on your system forever, *even* if you build a new image with an identical name. Building this new image simply moves the tag to the new image; it doesn't delete or replace the old image.

Consequently, as you iterate while you are creating a new image, you will often create many, many different images that take up unnecessary space on your computer. To see the images currently on your machine, you can use the `docker images` command. You can then delete tags you are no longer using.

Docker provides a tool called `docker system prune` for doing general cleanup. This will remove all stopped containers, all untagged images, and all unused image layers cached as part of the build process. Use it carefully.

A slightly more sophisticated approach is to set up a `cron` job to run an image garbage collector. For example, you can easily run `docker system prune` as a recurring `cron` job, once per day or once per hour, depending on how many images you are creating.

Summary

Application containers provide a clean abstraction for applications, and when packaged in the Docker image format, applications become easy to build, deploy, and distribute. Containers also provide isolation between applications running on the same machine, which helps avoid dependency conflicts.

In future chapters, we'll see how the ability to mount external directories means we can run not only stateless applications in a container, but also applications like MySQL and others that generate lots of data.

Previous chapter

< 1. Introduction

Next chapter

3. Deploying a Kubernetes Cluster >